

COUNTERSIGN — Frontend Brief

Owners: Frontend dev 1 (marketing + auth + shell) · Frontend dev 2 (application surface) **Stack:** React 18 + TypeScript (strict) · Vite · TanStack Query · React Router 6 · Tailwind + CSS custom properties · Framer Motion · D3-force · Recharts

Read `01-BACKEND-BRIEF.md` §3–§11 before writing a line of fetch code. That file is the API contract and it wins any disagreement.

1. What we're building and who it's for

COUNTERSIGN is a financial risk analyst for businesses too small to hire one. The user is a bookkeeper or owner-operator at a 5–50 person company. They are not a data scientist. They are interrupted constantly. They need to know two things fast: **what needs my attention, and why should I believe you.**

That second question is the entire product. Every screen has to answer it without being asked.

Two audiences, two surfaces:

- **Marketing site** (`/` , `/product` , `/security`) — this is what a Seed Round judge from Pear or Afore sees first. It has to look like a company, not a hackathon.
 - **Application** (`/app/*`) — this is what the technical judges will be clicking through. It has to look like a tool someone uses daily.
-

2. Design direction

2.1 The concept: ink and paper

The product's thesis is provenance — every claim traces to a document. So the interface is built on a literal material inversion: **the application chrome is ink, the evidence is paper.**

Everything that is *the system's opinion* (feed, graph, dashboards, controls) sits on deep archival ink. The moment you open a source document, the surface flips to paper — warm off-white, a text serif, generous measure, the cited span marked like a highlighter pass. That

transition is the memorable thing in this UI and it happens to be the exact moment the product proves itself. Spend the boldness there and keep everything else quiet.

This is not a dark-mode-with-a-neon-accent app. The reds are rubber-stamp reds, not cyberpunk. It should feel like a well-built audit tool, because that is what it is.

2.2 Palette — 6 tokens, no more

```
:root {  
  /* ink: the system's surfaces */  
  --ink-900: #0D1B2A; /* deepest ground — app background, nav */  
  --ink-700: #16293C; /* raised panels, cards, sheet backgrounds */  
  --ink-500: #2C4A64; /* borders, dividers, inactive strokes */  
  --ink-200: #A8BDD0; /* secondary text on ink, axis labels, metadata */  
  --ink-050: #E8EFF5; /* primary text on ink */  
  
  /* paper: the evidence surface */  
  --paper: #F7F4ED; /* document reader background */  
  --paper-text: #1A1712; /* document body text */  
  --paper-rule: #D8D0C0; /* margin rules, line separators in reader */  
  
  /* signal: routing severity, used ONLY for routing severity */  
  --stamp-red: #A8324A; /* escalate_now */  
  --stamp-amber: #B9782C; /* flag_for_review */  
  --stamp-slate: #5B7386; /* auto_file */  
  
  /* one interactive accent, used ONLY for interactive affordances */  
  --verify: #1F9C8B; /* focus rings, active toggles, primary buttons, confirmed */  
}
```

Colour discipline, enforced in review:

- The three `--stamp-*` values mean routing severity and nothing else. Never use `--stamp-red` for a delete button, a form error, or decoration. A red thing on screen always means "this insight is escalated."
- `--verify` means "you can interact with this" or "this was confirmed." Never decorative.
- Confidence and vacuity are **not** encoded in colour. They're encoded in position, length, and number. Colour is already spent on severity; overloading it makes the screen unreadable.
- No gradients anywhere except one: a 1px-to-transparent top edge highlight on raised panels, at 6% white. That's it.

2.3 Type

Two families, clearly distinct, plus a mono that is only ever used for actual machine identifiers.

Role	Family	Notes
UI + display	Instrument Sans (Google Fonts, variable)	Slightly condensed grotesque with real character. Not Inter — Inter is the default tell and half the projects at this hackathon will use it.
Document reader body	Literata (Google Fonts, variable)	A screen-reading text serif. Used <i>only</i> inside the paper surface. Its presence is what makes the reader feel like a document rather than a div.
Machine identifiers	JetBrains Mono	Byte offsets, UUIDs, request IDs, SHA prefixes, temperature values. Justified because these are literally code. Never used for labels, captions, or "technical flavour."

Type scale (1.25 ratio, rem, 16px base):

display-1	3.052rem / 1.05 / -0.02em	weight 600	hero headline only
display-2	2.441rem / 1.10 / -0.015em	weight 600	page titles
h1	1.953rem / 1.15 / -0.01em	weight 600	
h2	1.563rem / 1.25 / -0.005em	weight 600	
h3	1.25rem / 1.35 / 0	weight 600	
body	1rem / 1.55 / 0	weight 400	
body-sm	0.8rem / 1.50 / 0	weight 400	metadata, table cells
micro	0.64rem / 1.45 / 0.01em	weight 500	badge text, axis ticks
reader-body	1.125rem / 1.70 / 0	weight 400	Literata, paper surface
reader-sm	0.95rem / 1.65		Literata, invoice line items

Reader measure is capped at **66ch**. UI body copy at **72ch**.

Typographic prohibitions — these are the tells that make a page read as generated, and any of them showing up in review gets reverted:

- No all-caps labels. Not on badges, not on eyebrows, not on table headers. Sentence case with weight and colour doing the hierarchy work.
- No accenting a single word in a headline in a different colour or italic.

- No `→` appended to button or link text. The button says what happens: "Open document", "Run ablation", "Recalibrate now".
- No `A · B · C` middle-dot meta strings. Use real separators or real layout.
- No `WORD — fragment` spaced-em-dash label constructions.
- No eyebrow label above every heading. Only where the section genuinely needs categorising.

2.4 Layout system

8px spatial grid. Radii: `2px` on inputs and badges, `6px` on panels, `0` on the paper surface (paper has no rounded corners — that's part of why it reads as paper). One shadow token only, used exclusively on overlays and never on inline cards:

```
--shadow-overlay: 0 24px 48px -12px rgba(6, 14, 22, 0.55);
```

Inline panels are separated by `1px solid var(--ink-500)` at 40% opacity and background elevation, never by shadow. Uniform rounded cards with identical soft shadows is the SaaS-kit look and we're not doing it.

2.5 Motion

Tokens:

```
--dur-instant: 120ms; /* state flips: toggle, checkbox, badge change */
--dur-quick:   220ms; /* panel open, tooltip, hover */
--dur-move:    420ms; /* route transition, sheet slide, surface flip */
--dur-story:   900ms; /* the hero sequence, once */
--ease-out:    cubic-bezier(0.16, 1, 0.3, 1);
--ease-inout:  cubic-bezier(0.65, 0, 0.35, 1);
```

Rules:

1. **One orchestrated non-triggered moment on the whole site** — the hero sequence (§5.1). Everything else moves only in response to a user action.
2. **No fade-and-slide-up on section scroll.** That is the single most recognizable generated-page signature. Sections are simply there.
3. Motion that answers an action is required, not optional: the ablation confidence number must *animate* from 0.81 to 0.40 so the user sees the change happen. The ECE number must count down during recalibration. A static number replacing another static number

loses the entire point.

4. @media (prefers-reduced-motion: reduce) — all durations collapse to 0ms except opacity crossfades capped at 100ms. The hero sequence renders in its final state immediately. Test this; a judge may have it on.

3. Route map

/	Landing (hero, thesis, pipeline, evidence, security, CTA)
/product	Deeper walkthrough with the interactive sub-demo
/security	Security & data posture page
/login	Sign in
/register	Create account + organization
/app	→ redirect to /app/feed
/app/feed	Insight feed (default landing after auth)
/app/feed/:insightId	Feed with detail sheet open (deep-linkable)
/app/graph	Risk graph canvas
/app/graph/:entityId	Graph focused on an entity
/app/document/:docId	Paper reader
/app/document/:docId?span=:insightId	Reader scrolled to and highlighting a span
/app/evidence	Eval dashboards (fragility / routing / calibration tabs)
/app/voice	Voice console
/app/ingest	Upload + job progress
/app/settings	Profile, org, members (owner only)

Deep-linkability matters for the demo: we need to be able to jump straight to any screen if something breaks live. Every piece of view state that matters (open insight, active filters, selected entity, active eval tab) lives in the URL, not in component state.

4. Working against the backend

4.1 Mock mode — start here, hour 0

Backend publishes fixtures at hour 3 (01-BACKEND-BRIEF.md §12). Until then and alongside, run:

```
MOCK_MODE=1 MOCK_ERROR_RATE=0.1 uvicorn api.main:app --port 8000
```

Fixtures return with a randomized 80–400ms delay and a 10% injected error rate. **Build loading and error states from the first commit** — do not build the happy path and retrofit

them. The error rate is there specifically to force this.

Generate types from the live OpenAPI schema rather than hand-writing them:

```
npx openapi-typescript http://localhost:8000/api/v1/openapi.json -o src/api/schem
```

Re-run this whenever the backend says the contract changed. Hand-written types will drift and cost hours.

4.2 Client conventions

- **TanStack Query** for all reads. Query keys are arrays mirroring the URL: `['insights', filters], ['insight', id], ['graph', graphFilters], ['evals', 'fragility']`.
- Access token in memory only (a module-level variable + React context). **Never** `localStorage`, **never** `sessionStorage`. The refresh token is an `HttpOnly` cookie the frontend cannot see and must not try to read.
- Single `Axios/fetch` wrapper with a response interceptor: on `401 + code === "TOKEN_EXPIRED"`, call `POST /auth/refresh` once, queue and replay the failed requests, then continue. On `401 + code === "REFRESH_REUSED"`, wipe client state and hard-redirect to `/login` with a notice that the session was ended for security.
- All requests `credentials: 'include'` so the refresh cookie travels.
- Every error toast shows the `request_id` in mono. During the demo that's how we diagnose anything in ten seconds instead of two minutes.

4.3 Error code handling — required behaviour per code

<code>error.code</code>	UI behaviour
<code>VALIDATION_FAILED</code>	Map <code>details.fields</code> onto the specific form inputs. No generic "something went wrong."
<code>TOKEN_EXPIRED</code>	Silent refresh + retry. User sees nothing.
<code>REFRESH_REUSED</code>	Hard logout, redirect to login, explain why.
<code>ACCOUNT_LOCKED</code>	Show countdown to <code>details.locked_until</code> . Disable the submit button until then.
<code>FORBIDDEN</code>	Don't show the error — the affordance should never have rendered. If it fires, that's a permissions-gating bug on our side.

	Log it.
RATE_LIMITED	Disable the triggering control for <code>details.retry_after_seconds</code> with a live countdown on the button itself.
NEMOTRON_UNAVAILABLE	Degrade, don't break. Feed still renders; escalated insights show a "reviewed classically" marker; a single dismissible banner explains the upstream is down.
VOICE_UNAVAILABLE	Automatically fetch <code>GET /voice/fallback/briefing</code> and play that. Show a small "playing recorded briefing" note. No modal, no blocking error.
PIPELINE_FAILED	Job card shows the failed stage from <code>details.stage</code> with a retry button.
network timeout > 8s on voice	Same as <code>VOICE_UNAVAILABLE</code> . Don't wait forever on stage.

4.4 Permission gating

`GET /auth/me` returns `permissions: string[]` . Gate on those strings, never on `role === 'owner'` .

```
const { can } = useAuth();
{can('calibration:run') && <RecalibrateButton />}
{can('ablation:run')
  ? <AblateButton />
  : <AblateButton disabled title="Ask an analyst to run this" />}
```

Permission strings in use: `insights:read` , `graph:read` , `documents:upload` , `ablation:run` , `voice:use` , `evals:read` , `calibration:run` , `users:manage` .

5. Marketing surface (Frontend dev 1)

5.1 Landing hero — the one orchestrated moment

Do not build a big-number-plus-gradient hero. The most characteristic thing in this product's world is **a claim being pulled out of a document and proving where it came from**. So that's the hero, animated once on load, then frozen and replayable.

Layout: two columns on $\geq 1024\text{px}$, stacked on mobile. Left is the headline and subhead. Right is a fixed-size paper card showing three lines of a real invoice.

The sequence (`--dur-story` , staggered, total $\sim 2.6\text{s}$, runs once):

t=0ms	Headline sets in place. No fade-up. It's just there, then weight settles from 500→600 over 300ms. Subtle. Reads as ink drying.
t=400ms	Paper card is already visible. A cited span within the invoice text gets a highlighter pass – a background wipe left-to-right over 280ms in a translucent amber, like a marker stroke.
t=900ms	A hairline connector draws from the highlighted span out to the right edge of the card (stroke-dashoffset animation, 320ms).
t=1300ms	An insight chip materializes at the connector's end: "Meridian Supply LLC →wired funds to→ Advent Holdings" (chip scales 0.96→1 and opacity 0→1 over 200ms)
t=1600ms	Confidence reads out: a number counts 0 → 81 over 500ms, with the label "confidence" beneath it. Then, 200ms later, a second smaller line appears: "traced to line 14, characters 412–501" in mono.
t=2600ms	Done. A quiet "Replay" text button appears under the card.

That last line — the byte offsets in mono — is the whole pitch. Nobody else's landing page will show you a character range.

Headline copy (no single-word colour accents):

Every number in this report can prove where it came from.

Subhead:

COUNTERSIGN reads your invoices, vendor mail, and news feeds, and tells you what needs attention. Every claim traces back to the exact sentence it came from. Nothing is generated.

Primary CTA: "See it on real documents" → `/product` **Secondary:** "Sign in" → `/login`

5.2 Landing sections, in order

1. **Hero** (above).
2. **The problem** — three short statements in a row, no cards, no icons, separated by vertical hairlines. "40 invoices a week. / Nobody reads them for fraud. / Enterprise

tooling starts at five figures."

3. **How it works** — the pipeline, as five numbered steps. **Numbering is justified here because it genuinely is a sequence** (extract → score → gate → escalate → brief). This is the one place numbered markers belong. Horizontal on desktop with connecting rules, vertical on mobile.
4. **The evidence section** — this is the section that wins the Seed Round and Xtract judges. Four claims, each with a real number pulled live from `GET /api/v1/evals/*` on a public read-only endpoint (or baked at build time from the fixtures if we don't want a public endpoint):
 - uncertainty ↔ fragility Spearman correlation
 - cascade escalation rate + LLM calls avoided
 - routing macro-F1 across N labeled cases
 - ECE before / after recalibration

Present as a 2x2 of statements with the figure set large in Instrument Sans and the method in one sentence beneath. **Include the failure we found**, in plain language, as a fifth item spanning the row. A landing page that voluntarily shows you its own failure case is a strong signal and no other team will do it.

5. **The paper moment** — a live, scrubbable mini version of the citation reader. Reader surface flips from ink to paper right there in the page. This is the sub-page demo: a real document, three insights, click one and watch the span light up.
6. **Security posture** — six one-line facts (argon2id, rotating refresh with reuse detection, org-scoped queries, synthetic data only, no keys in the browser, rate limited). Links to `/security`.

7. CTA + footer.

5.3 `/product` — the interactive sub-demo

A guided, four-step walkthrough of a single real case, controlled by the user, not autoplaying. Each step is a state of the same canvas rather than a new section:

1. **Ingest** — documents land, entity tags appear inline in the text.
2. **Graph** — entities assemble into a small force graph; the three-node ownership cycle resolves and gets emphasized.
3. **Gate** — a strip of insights sorts by vacuity; the low-uncertainty ones fold away into an

"auto-filed, 186 items" summary and the high-uncertainty tail stays. Then Nemotron's decision stamps onto the remaining ones. The point being made visually: *we only spent the LLM on the short list*.

4. **Prove it** — ablation. Zero the edge, watch confidence fall. Then the citation opens on paper.

Progress is a segmented control, not dots. Users can jump to any step — the judges will want to skip straight to step 4.

5.4 /security

Plain, dense, credible. Table of controls with a one-line "what it does" and "why it matters." Explicit statement: **all data in this demo is synthetic or from public sources; no real account numbers, credentials, or financial records are used anywhere.** Both the Compound and Xtract tracks have that as a hard rule and stating it unprompted is free credibility.

5.5 Auth screens

Single-column, centred at 420px, on `--ink-900` with a single raised panel. No decorative illustration, no split-screen marketing panel.

- `/login` — email, password, "Sign in". Inline field errors from `details.fields`. Lockout countdown rendered on the disabled button when `ACCOUNT_LOCKED`.
- `/register` — email, password, confirm, organization name. **Live password requirement checklist** (≥ 12 chars, not a common password) that ticks as they type. Requirements are enforced server-side; the checklist is a courtesy, so it must match the server rules exactly or users will be told a compliant password is invalid.
- Both: submit disabled while pending, button label changes to "Signing in...", never a spinner replacing the label entirely (layout shift).
- Post-auth redirect honours `?next=` if present, else `/app/feed`.
- Password managers must work: correct `autocomplete` attributes (`email`, `current-password`, `new-password`), real `<label>` elements, no synthetic input hacks.

6. Application shell (Frontend dev 1)

Persistent left rail, 72px collapsed / 224px expanded, state persisted in a cookie (not `localStorage` — we've banned it for tokens and it's simpler to just not use it at all).

Rail items: Feed, Graph, Evidence, Voice, Ingest, Settings. Each with a count badge where meaningful (Feed shows `by_routing.escalate_now` from `/insights/stats`, in `--stamp-red`).

Top bar: organization name, active scenario name, a global search input (queries `/insights?q=`), and the account menu. Right side holds a **live job indicator** — when an ingest job is running, a thin determinate progress bar spans the full width beneath the top bar, driven by the websocket. When it completes, it does not toast; the feed count just updates. Toasts for background success are noise.

Route transitions: `--dur-move`, opacity + 8px translate on the content region only. The rail and top bar never move.

7. Insight feed (Frontend dev 2)

7.1 Layout

Two panes on $\geq 1280\text{px}$: filter sidebar (240px) | feed list. The detail view is a **right sheet** at 560px overlaying the list, not a route replacement, so the list stays visible and the user keeps their place. On $< 1280\text{px}$ the sheet is full-screen and the filters collapse into a sheet trigger.

7.2 Header strip

From `GET /api/v1/insights/stats`:

- Three routing counts as a single horizontal stacked bar, proportional, with counts labeled. Clicking a segment filters the feed. `--stamp-red` / `--stamp-amber` / `--stamp-slate`.
- "28 of 214 insights needed a language model" — the cascade fact, stated in words on the main screen. This is the Nemotron track's thesis and it should be visible without navigating to the eval page.
- Documents ingested count.

7.3 Row anatomy

Each row is a horizontal record, not a card. Hairline separators, no per-row shadow, no border radius, 12px vertical padding, hover changes background one elevation step only.

Meridian Supply LLC	wired funds to	Advent Holdings	
"Payment of \$48,200 was routed through Advent Holdings on behalf of..."			
INV-4471 · 14 Sep · escalated · nemotron	conf 0.81		
	vac 0.62		

↑ 3px left bar in the routing severity colour

- Subject and object are the visual anchors, in `h3` weight; the relation verb sits between them at `body-sm` in `--ink-200`.
- Citation preview is one line, truncated with a real ellipsis, in Literata at `body-sm` — a small foreshadowing of the paper surface.
- Confidence and vacuity as two thin horizontal meters (4px tall, right-aligned, fixed 120px track). Numeric value always adjacent. Meters are `--ink-050` on `--ink-500` track; **not colour-coded**, because colour means severity.
- `resolved_by: "nemotron"` gets a small outlined marker. `degraded: true` gets a hairline-outlined "classical fallback" marker.
- Row is a `<button>` or link with a visible `--verify` focus ring. Keyboard: `j / k` or arrows to move, `Enter` to open, `Esc` to close the sheet.

7.4 Filters

All filter state is URL search params, so any filtered view is shareable and reload-safe.

- Routing: three checkboxes (`?routing=escalate_now&routing=flag_for_review`)
- Resolver: segmented `all / classical / nemotron`
- Confidence: dual-handle range slider → `min_confidence , max_confidence`
- Vacuity: single-handle minimum → `min_vacuity`
- Relation: multi-select of the six relation types
- Sort: `-created_at` (default), `-confidence` , `confidence` , `-vacuity` , `-fragility`
- Free text: `q`

Debounce text and slider input at 300ms. Show a "clear all" when any filter is active, with the active filter count.

Infinite scroll via cursor pagination (`pagination.next_cursor`), using TanStack Query's `useInfiniteQuery`. Intersection observer sentinel, 400px root margin. Keep a "load more" button as a fallback for keyboard users — pure infinite scroll is an accessibility trap.

7.5 Detail sheet

Sections, top to bottom:

1. **Claim** — subject / relation / object, large. Entity names are links to `/app/graph/:entityId`.
2. **Trust** — four values: confidence, vacuity, dissonance, fragility. Present as a small 4-row table with numeric values and meters. `fragility` may be `null` before fuzzing — render "not yet measured", never `0` or `-` without explanation.
3. **Source** — the cited sentence rendered in Literata on a `--paper` inset block, with the byte range in mono beneath it. A primary button: "Open full document" → `/app/document/:docId?span=:insightId`.
4. **Routing** — the bucket, who decided it, and if `resolved_by === 'nemotron'`, the `nemotron.rationale` text in a quoted block with the latency in mono. Label it clearly as the model's stated reasoning, not as fact.
5. **Attention & ablation** (§8) — only when `attention_available`.
6. **Fragility trials** — a compact table from `fragility_trials`: perturbation, whether the label flipped, confidence delta. Five rows. This is small and quiet and a technical judge will stop on it.
7. **Graph context** — a 240px-tall mini force graph of `graph_neighborhood`, clickable through to the full canvas.

Sheet opens with `--dur-move` slide + the list dimming 10%. Deep-linked via `/app/feed/:insightId` so it survives a reload.

8. Ablation panel — the causal explainability moment (Frontend dev 2)

This lives inside the detail sheet and it is one of the two or three things judges will remember. Build it carefully.

Before state. The cited sentence rendered token-by-token from `tokens[]`. Attention edges from `attention[]` drawn as arcs *above* the token row (SVG, quadratic Béziers), stroke width scaled to `weight`, stroke `--ink-200` at 30–90% opacity by weight. Hovering an arc raises it and dims the others.

Interaction. Click an arc to mark it for masking. Marked arcs render as dashed in `--`

`verify`. Multi-select supported (`masked_edges` is an array). A mode toggle: "Zero attention" / "Uniform attention". Then a primary button: "Run ablation".

Request. `POST /api/v1/ablation/insights/:id with { masked_edges, mode }.`

After state — this is where the motion budget gets spent. On response:

1. The masked arc visibly breaks: it animates to `stroke-dashoffset` full and fades to 15% over `--dur-quick`.
2. The confidence number **counts** from `before.confidence` to `after.confidence` over 600ms with `--ease-out`, and its meter shrinks in sync. Never snap-replace the value; the animation *is* the evidence.
3. Vacuity counts up in parallel.
4. If `delta.routing_changed`, the routing badge does a stamp transition: scales to 1.08 and settles back at 1.0 over 240ms while its colour crossfades from the old severity to the new. It should read like a rubber stamp landing.
5. The `interpretation` string renders beneath in `body`. It is server-templated — render it as-is, do not reword or reformat it.
6. A verdict line: `load_bearing === true` → "This connection is causally responsible for the flag." `false` → "Removing this connection barely changed the result — the flag does not depend on it." Both are true and interesting; don't hide the negative case.

Ablation history. `ablation_history[]` renders as a compact list of prior runs with their deltas, so repeated runs accumulate visible evidence rather than overwriting.

Error handling. `RATE_LIMITED` → disable the button with a countdown (30/min limit, a judge mashing the button will hit it). `500` → keep the before-state intact and show the error inline; never leave the panel in a half-ablated visual state.

9. Risk graph (Frontend dev 2)

9.1 Rendering

`d3-force` for layout, **SVG** for rendering (≤ 300 nodes per `limit_nodes`, so SVG is correct here and gives us free hit-testing and accessibility; canvas would be premature). Forces: `forceLink` with distance inversely proportional to edge `confidence`, `forceManyBody` at -280, `forceCollide` at node radius + 4, `forceCenter`.

Run the simulation in a web worker if the main thread stutters; at 300 nodes it probably

won't, so don't do this preemptively.

9.2 Encoding

Data	Visual
<code>node.entity_type</code>	shape — ORG = square, PERSON = circle, ACCOUNT_REF = diamond, other = small circle
<code>node.risk</code>	radius, 6px → 20px, sqrt scale
<code>node.mention_count</code>	fill opacity, 40% → 100%
<code>node.flags</code>	a hairline outer ring, <code>--ink-200</code>
<code>edge.relation</code>	stroke pattern — solid for funds movement, dashed for ownership, dotted for shared attributes
<code>edge.weight</code>	stroke width, 1px → 4px
<code>edge.routing</code>	stroke colour, the three <code>--stamp-*</code> values
<code>edge.confidence</code>	stroke opacity, 35% → 100%
<code>cycles[]</code>	the cycle's edges get a 2px halo in <code>--stamp-red</code> at 25%, and the cycle is listed in a side panel

Cycles come from the server (`cycles[]`). Do not attempt cycle detection client-side; the backend runs Tarjan SCC and the answer is authoritative.

9.3 Interaction

- Zoom/pan via `d3-zoom`, scale extent 0.4–4. A "reset view" control, and "fit to content" on first load.
- Hover a node: tooltip with canonical name, type, risk, degree, flags. Incident edges raise, everything else drops to 20% opacity.
- Click a node: `GET /graph/entities/:id`, open a left detail panel with aliases, neighbours, documents, insight count. URL becomes `/app/graph/:entityId`.
- Click an edge: opens the feed detail sheet for its first `insight_ids[0]` — the graph and the feed are two views of one dataset and must feel connected.
- Filter panel: entity type, relation type, min confidence, routing. Also `root_entity_id +`

`depth` (1–3) for neighbourhood mode.

- `truncated: true` → a visible notice with the node cap and a control to raise `limit_nodes`. Silently hiding data in a forensics tool is unacceptable.
- **Cycle callout:** when `cycles.length > 0`, a panel lists each cycle with its length and risk and a "focus" button that isolates it. On the demo scenario this is the three-node ownership loop and it should be trivially findable — a judge should not have to hunt for the fraud we planted.

9.4 Accessibility

A force graph is not screen-reader navigable, so provide a real alternative: a "table view" toggle that renders the same nodes and edges as two sortable tables. Not a stub — the same filters apply and rows link to the same details. This also happens to be genuinely useful for anyone who prefers it.

10. Paper reader (Frontend dev 2)

The surface flip. This is the one place the whole visual system inverts and it should feel like picking up a piece of paper.

10.1 Transition

Route `/app/document/:docId`. On enter: the ink background crossfades to `--paper over -dur-move` while the content column narrows to 66ch and the body font shifts to Literata. The left rail stays ink — the document is an object *in* the app, not a new app.

10.2 Content

Render `raw_text` from `GET /api/v1/documents/:id`.

Hard rule: `raw_text` goes into `textContent`, never `innerHTML`, never through a markdown renderer. Two reasons, both non-negotiable: it's untrusted document content, and every character offset in the system indexes into this exact string. Normalizing whitespace or trimming breaks every citation in the product.

Build highlight spans by sorting `spans[]` by `char_start` and slicing `raw_text` into alternating plain and highlighted segments. Handle overlapping spans by splitting at boundaries — two insights citing overlapping ranges is realistic and must not crash the renderer. Write a unit test for the overlap case.

- Highlighted spans: translucent background in the span's routing severity colour at 18%,

with a 2px bottom border at 60%. Not a block fill — it should read as a marker pass over text.

- A left margin rail shows a tick per span at its vertical position, colour-coded by routing. Clicking a tick scrolls to it. This is the document's minimap.
- `?span=:insightId` scrolls that span into view at 30% viewport height and pulses its background once (18% → 34% → 18% over 700ms). One pulse, not a loop.
- Hovering a span shows a small inline chip with the relation and confidence, and "Open insight" which opens the feed sheet over the paper.
- Entity mentions from `mentions[]` get a subtle dotted underline in `--paper-rule` with a tooltip on hover showing type and tagger confidence. Distinct from citation highlights — mentions are what we tagged, citations are what we concluded.

10.3 Controls

Top of the reader, quiet: document title, source type, received date, a toggle for "show entity mentions", a toggle for "show only escalated spans", and a back control that returns to wherever the user came from (feed or graph) with their state intact.

11. Evidence dashboards (Frontend dev 2)

Route `/app/evidence` with three tabs, tab in the URL (`?tab=fragility`). **Recharts** for all of these — it's in the stack, it's declarative, and at this data volume the performance is fine.

11.1 Fragility tab — `GET /api/v1/evals/fragility`

The most important screen in the app for a technical judge.

Primary: scatter plot. `x = vacuity`, `y = fragility`, one point per insight from `scatter[]`. Point colour = routing severity. 214 points. Add a fitted trend line. Annotate the Spearman correlation, Pearson, and p-value in a corner block with the coefficient large and the method beneath it. Axes labeled "epistemic uncertainty (vacuity)" and "measured fragility under perturbation" — the full words, because the point is legibility to a judge who has 90 seconds.

Clicking a point opens that insight's detail sheet. The chart is navigable, not decorative.

Secondary: quartile table. From `quartile_table[]`. Four rows: quartile, vacuity range, mean fragility, flip rate. Render the flip rate as a number *and* a small inline bar. This table is the cleanest single statement of the project's thesis — the bottom-quartile vs top-quartile

flip rate contrast should be impossible to miss.

Tertiary: per-perturbation bar chart. From `by_perturbation[]` . Grouped bars: flip rate, mean absolute confidence delta, relation loss rate, across the five perturbation families. Tells the "which attacks hurt us" story.

Interpretation block. Render the server's `interpretation` string prominently at the top in `h3` . It's the sentence we want a judge to read if they read nothing else.

11.2 Routing tab — `GET /api/v1/evals/routing`

Confusion matrix as a 3×3 heatmap. Cell fill opacity by count, count centred in the cell, row/column totals in a gutter. Diagonal cells outlined in `--verify` . Axis labels: "ground truth" (rows) and "predicted" (columns), spelled out — unlabeled confusion matrices are a classic own-goal.

Per-class metrics table from `per_class[]` : bucket, precision, recall, F1, support. Macro-F1 and accuracy called out above it.

Cascade comparison from `cascade_baseline` — the argument for why Nemotron is in the pipeline at all. Three bars: classical-only accuracy, cascade accuracy, Nemotron-on-everything accuracy. Beneath them, the LLM call counts (28 vs 214) as a second paired bar. The shape of these two charts together *is* the Nemotron track submission: nearly all the accuracy, a fraction of the calls. Render `cascade_baseline.interpretation` under it.

Documented failures from `documented_failures[]` . Give this real space, not a collapsed accordion. Per failure: the sentence in Literata on a paper inset, ground truth vs predicted as two badges, and the full `note` text in `body` at comfortable measure. Header: "Where it gets things wrong." Voluntarily surfacing this is exactly what the track asked for and it should look deliberate, not buried.

Nemotron audit log — paginated table from `GET /api/v1/routing/runs` : timestamp, insight link, decision, rationale (truncated, expandable), latency, token counts. Prompt SHA in mono. This is the "show your work" artifact.

11.3 Calibration tab — `GET /api/v1/evals/calibration`

Reliability diagram. From `snapshots[].bins` . `x = avg_conf` , `y = accuracy` , the `y=x` diagonal drawn as a dashed reference in `--ink-500` . Two series overlaid: baseline and post-recalibration, distinguished by line style and a legend (not by colour alone). Bin `count` drives point size. The visual story is the post-recalibration line hugging the diagonal more closely.

Metrics. ECE, MCE, Brier for both snapshots, in a before/after pair with the delta. ECE gets

the largest treatment — it's the number we're claiming to improve.

The recalibrate control. Owner-only (`can('calibration:run')`). A single primary button: "Recalibrate now".

The interaction, which is the demo finale and must be rehearsed:

1. Press → button enters pending state, label "Recalibrating...", the reliability chart dims to 40%.
2. Response arrives (backend targets <4s). The ECE figure **counts down** from `before.ece` to `after.ece` over 900ms. Simultaneously the post-recalibration series animates onto the chart, drawing left to right.
3. The temperature value appears in mono: `T = 1.00 → 1.37`.
4. A single line states the improvement: "Calibration error fell 56% against 43 logged hard cases."
5. Send an `Idempotency-Key` header (a UUID generated when the button mounts) so a double-click returns the same snapshot instead of running twice. A judge will double-click.

If the response takes longer than 6s, show a determinate-ish progress affordance rather than an indefinite spinner. Dead air on stage is worse than a slow bar.

12. Voice console (Frontend dev 2)

Route `/app/voice`. Two halves: briefing on top, ask below.

12.1 Briefing

Controls: scope segmented control (`flagged / escalated / all_new`), max items stepper (1–5), and a primary "Play briefing" button.

`POST /api/v1/voice/briefing` → render a native-behaviour custom audio player using a hidden `<audio>` element with `audio_url`. Custom transport (play/pause, seek, 0.75×/1×/1.25× speed) because the native control is ugly and we need the transcript sync anyway.

Transcript sync is the thing that sells the voice track. Render `transcript[]` as a list of segments. On `timeupdate`, highlight the active segment by comparing `currentTime * 1000` against `start_ms / end_ms`. When the active segment has a non-null `insight_id`:

- that segment's text gets a `--verify` left border and elevated background
- the corresponding insight card in a right-hand column highlights in sync
- clicking any segment seeks the audio to its `start_ms`

So the judge hears "Meridian Supply routed a payment through Advent Holdings, confidence eighty-one percent" while watching that exact insight light up with its citation. That's the demo beat.

Waveform: render a simple static amplitude bar strip (48 bars, derived from a decoded `AudioBuffer`, or a fixed decorative-but-honest placeholder if decoding is too slow) with a playhead. Don't animate bars to fake a visualizer — a fake visualizer in a product about provenance is a bad joke.

12.2 Ask

A push-to-talk button — hold to record via `MediaRecorder (audio/webm; codecs=opus)`, release to send. Show elapsed time and a hard 30s cap with a visible countdown from 25s. Handle the mic permission denial path explicitly with instructions, not a dead button.

POST `/api/v1/voice/ask` (multipart) → render, in this order:

1. **What it heard** — `heard`, with `stt_confidence` shown as a small number. If `stt_confidence < 0.7`, add "Didn't catch that clearly?" and a re-record affordance. Showing the transcription honestly is better than pretending it never mishears.
2. **Intent** — `intent` as a small marker. If `unknown`, the answer is a rephrase prompt and no citation block renders.
3. **Answer** — `answer_text` in `body`.
4. **Citation** — `citation.sentence_text` in Literata on a paper inset, with document title and the byte range in mono, and "Open document" linking to `/app/document/:doc_id?span=:insightId`.
5. **Ablation reference** — if `ablation_run_id` is present, a link to that insight's ablation history.
6. Audio auto-plays the response from `audio_url`.

Keep a rolling session history of Q&A pairs above the input so the judge can see the sequence of questions asked.

12.3 Fallback — build this before the happy path is polished

If `POST /voice/briefing` returns `VOICE_UNAVAILABLE`, times out at 8s, or errors at all: **automatically** fetch `GET /api/v1/voice/fallback/briefing` and play it, with a small honest note — "Playing recorded briefing; live synthesis is unavailable." No modal. No retry dialog. The demo continues.

Additionally, preload the fallback audio on mount so it's in the browser cache before we need it. Conference wifi is the single most likely thing to break this project on stage.

13. Ingest screen (Frontend dev 1)

- Drop zone accepting `.txt`, `.csv`, `.pdf`, `.eml`, max 50 files, 2 MB each. Validate client-side *and* handle the server's `PAYLOAD_TOO_LARGE` / `UNSUPPORTED_MEDIA` — client validation is a courtesy, the server is the authority.
 - Source type selector (the `doc_source` enum).
 - **"Load demo scenario" panel** — three buttons for `meridian_shell_ring`, `clean_baseline`, `invoice_flood`, each with a one-line description of what it contains. This is the button we press on stage. Make it the most prominent thing on the page, not hidden in settings.
 - Job progress from `WS /api/v1/ws/jobs/:jobId`, with **REST polling at 2s as a fallback if the socket doesn't open within 3s**. Render the five pipeline stages from `stage_progress` as a segmented bar, each stage filling in turn, `docs_done` / `docs_total` and a running `insights_found` count.
 - `duplicates_skipped > 0` → state it plainly.
 - On `state: "failed"` → show `details.stage` and a retry control.
 - On completion: navigate to the feed automatically. The judge shouldn't have to find it.
-

14. Component inventory

Build these as a small primitive layer first (hour 0–3) so both devs compose rather than duplicate.

Primitives: `Button` (primary/secondary/quiet/danger, pending state built in), `IconButton`, `Input`, `Select`, `Checkbox`, `RangeSlider` (single + dual), `SegmentedControl`, `Badge`, `Meter`, `Tooltip`, `Sheet`, `Dialog`, `Tabs`, `Table` (sortable), `Toast`, `Skeleton`, `EmptyState`, `ErrorState`, `CountUp`.

Domain components: `RoutingBadge` , `ConfidenceMeter` , `VacuityMeter` , `TrustPanel` , `CitationBlock` , `PaperSurface` , `InsightRow` , `InsightSheet` , `AttentionArcs` , `AblationPanel` , `GraphCanvas` , `GraphTableView` , `EntityPanel` , `CycleCallout` , `DocumentReader` , `SpanMinimap` , `ScatterFragility` , `ConfusionMatrix` , `ReliabilityDiagram` , `CascadeComparison` , `FailureCaseCard` , `NemotronAuditTable` , `RecalibrateControl` , `BriefingPlayer` , `TranscriptSync` , `PushToTalk` , `JobProgress` , `ScenarioLoader` .

`CountUp` is small and gets used in four places (ablation confidence, ECE, hero confidence, insight counts). Write it once, respect `prefers-reduced-motion` inside it, and never hand-roll a second one.

15. State management

Match the tool to the problem. No Redux. No Zustand for things the URL should hold.

State	Where
Server data (everything from the API)	TanStack Query. It is the cache; don't mirror it into local state.
Filters, sort, pagination, active tab, selected insight/entity	URL search params + route params. Deep-linkable and reload-safe, which we need for demo recovery.
Auth session (user, permissions, access token)	One React context, token in a module variable, never persisted.
Ephemeral UI (sheet open, rail collapsed, hovered node)	Local <code>useState</code> .
Job progress websocket	A single custom hook (<code>useJobProgress</code>) that writes into the Query cache via <code>setQueryData</code> so the rest of the app reads it like any other server state.
Audio playback position	Local state in the player component; do not lift it.

Cache config: `staleTime` 30s for `/insights` and `/graph` , 5 minutes for `/evals/*` (they only change when we re-run a script), 0 for `/ingest/jobs/*` . Invalidate `['insights']` and

['graph'] on job completion.

16. Performance

- Route-level code splitting: marketing, app shell, graph, evidence, voice as separate chunks. The graph chunk drags in `d3-force` and the evidence chunk drags in `Recharts` — neither should be in the landing page bundle. Target landing route JS under 180 KB gzipped.
 - `React.memo` on `InsightRow` and virtualize the feed with `@tanstack/react-virtual` once the list exceeds ~200 rows. Below that, don't bother.
 - Memoize the D3 simulation setup on node/edge identity, not on the array reference — re-running `forceSimulation` on every render will visibly jitter the graph and is the most likely performance bug in this project.
 - Self-host the three font families as subsetting `woff2` with `font-display: swap`. No render-blocking Google Fonts request; the hero must paint immediately.
 - Preload the fallback briefing audio and the demo scenario's document text on app mount.
 - No layout shift on data load: skeletons must occupy the same dimensions as the loaded content. Measure with CLS in Lighthouse before we call it done.
-

17. Accessibility

The quality floor, built in rather than retrofitted:

- Every interactive element reachable by keyboard with a **visible** `--verify` focus ring (2px offset). Never `outline: none` without a replacement.
- Colour is never the sole carrier of meaning. Routing severity has colour *and* a text label. Chart series are distinguished by line style *and* legend.
- Contrast: `--ink-050` on `--ink-900` and `--paper-text` on `--paper` both clear AA for body text. Verify `--ink-200` on `--ink-700` for metadata — if it fails AA at `body-sm`, lighten it rather than shrinking usage.
- Sheets and dialogs: focus trapped, `Esc` closes, focus returns to the trigger, `aria-modal` and a labelled heading.

- The force graph has the table-view alternative (§9.4). The charts have accessible data tables behind a disclosure.
 - Audio: transcript is always visible, never behind a toggle. That's both an accessibility requirement and the feature that makes the briefing legible to a judge in a loud room.
 - Live regions: job completion and ablation results announce via `aria-live="polite"`. Errors via `role="alert"`.
 - `prefers-reduced-motion` fully respected, including the hero sequence and every `CountUp`.
 - Responsive to 375px. The feed, reader, and voice console must all be usable on a phone — a judge may pull this up on their own device.
-

18. Security responsibilities on the frontend

- **No token in `localStorage` or `sessionStorage`.** Access token in memory, refresh token in an `HttpOnly` cookie we never touch.
 - **No API keys in the frontend.** Not ElevenLabs, not NVIDIA, not anything. All upstream calls go through our backend. If a task seems to need a key in the browser, the design is wrong — escalate to Faaz.
 - **`raw_text` and all document-derived strings render as text nodes, never `innerHTML`.** No `dangerouslySetInnerHTML` anywhere in this codebase. If a PR contains it, reject the PR.
 - Validate and clamp anything used as an index — a malformed `char_start` should produce a missing highlight, not a crashed reader.
 - All forms submit via explicit handlers. No native `<form>` uncontrolled submission paths that bypass validation.
 - `target="_blank"` links carry `rel="noopener noreferrer"`.
 - `.env` files are gitignored; only `VITE_API_BASE_URL` is ever exposed to the client bundle, and treat even that as public.
 - Don't log response bodies to the console in production builds — document text is in there.
-

19. Definition of done — frontend

Marketing

- Hero sequence runs once on load, completes under 3s, has a working replay, and renders in final state under `prefers-reduced-motion`
- Evidence section shows four real numbers plus the documented failure
- `/product` four-step demo is user-controlled, jumpable to any step, and step 4 (ablation + citation) works standalone
- `/security` states the synthetic-data-only policy explicitly
- Lighthouse: performance ≥ 90 , accessibility ≥ 95 on `/`

Auth

- Register $\rightarrow$ auto-login $\rightarrow$ `/app/feed` works end to end
- Field-level errors render from `details.fields`
- `ACCOUNT_LOCKED` shows a live countdown
- Silent refresh works: leave the app idle past 15 minutes, come back, no re-login, no visible flicker
- `REFRESH_REUSED` hard-logs-out with an explanation
- Password managers autofill both screens correctly

Feed

- Renders 200+ insights with no visible jank
- Every filter is in the URL and survives reload
- Deep link `/app/feed/:insightId` opens the sheet directly
- `fragility: null` renders as "not yet measured"
- Keyboard navigation: `j / k`, `Enter`, `Esc` all work
- Empty state (no insights) and error state both designed, not default

Ablation

- Clicking an attention arc marks it; multi-select works
- Confidence number animates rather than snapping

- Routing change triggers the stamp transition
- `load_bearing: false` renders its own honest verdict
- Rate limit disables the button with a countdown
- History accumulates across runs

Graph

- 300 nodes render and pan/zoom smoothly
- Server-provided cycles are visually obvious without hunting
- Node click → entity panel; edge click → insight sheet
- `truncated: true` surfaces a visible notice
- Table view is fully functional, not a stub

Reader

- Surface flip from ink to paper is smooth and complete (background, font, measure)
- Highlight offsets are pixel-correct against `raw_text` — verified on all three seed scenarios
- Overlapping spans render correctly (unit tested)
- `?span=` scrolls to and pulses the right span, once
- Zero `dangerouslySetInnerHTML` in the repo (grep it)

Evidence

- Scatter plot renders 214 points with the trend line and correlation block
- Quartile table makes the top-vs-bottom contrast unmissable
- Confusion matrix axes are labeled with words
- Cascade comparison shows both accuracy and call-count bars
- Documented failures are given real space with the full note text
- Recalibrate: ECE counts down, chart series draws in, temperature shows, idempotent under double-click
- Recalibrate button hidden entirely for non-owners

Voice

- Briefing plays; transcript segments highlight in sync with audio
- Active segment's `insight_id` highlights the matching insight card
- Clicking a segment seeks the audio
- Push-to-talk records, sends, and renders `heard` + `answer` + `citation`
- Mic permission denial path has real instructions
- **Fallback briefing plays automatically on `VOICE_UNAVAILABLE` or 8s timeout, and the fallback audio is preloaded on mount**

Cross-cutting

- TypeScript strict, zero `any` in `src/`, types generated from the live OpenAPI schema
 - Every one of the 15 error codes in §4.3 has its specified behaviour, exercised via `MOCK_ERROR_RATE=1`
 - Usable at 375px width on feed, reader, and voice
 - `prefers-reduced-motion` honoured everywhere including `CountUp`
 - No `localStorage` / `sessionStorage` usage anywhere (grep it)
 - Production build served from the backend container works end to end
-

20. Division of work

Frontend dev 1 — design token implementation, primitive component layer, landing page and hero sequence, `/product` walkthrough, `/security`, both auth screens, app shell and rail, ingest screen, settings, protected routing and the auth/refresh interceptor.

Frontend dev 2 — insight feed and filters, detail sheet, attention arcs and ablation panel, graph canvas and table view, entity panel, paper reader and span rendering, all three evidence dashboards, recalibration control, voice console and transcript sync.

Shared, built jointly in hours 0–3 before splitting: tokens, type scale, primitives, API client + interceptor, generated types, `CountUp`, `PaperSurface`, `RoutingBadge`, `CitationBlock`. Getting these right together is what keeps the two halves of the app looking like one product.

Integration checkpoints: hour 6 (shell + primitives merged), hour 12 (feed + reader against real API), hour 18 (evidence + voice against real API), hour 22 (full demo walkthrough)

rehearsed end to end, twice).